

Projeto Cofre Eletrônico

Vitor Souza de Oliveira - 169244

Breno de Moraes Gobato - 168504

Flávio Henrique Santos Ferreira - 168690

Link código: <https://github.com/vitorSouzaUNIFESP/ProjetoAOC>



1. Resumo

Este projeto consiste no desenvolvimento de um **Simulador de Cofre Eletrônico** utilizando a linguagem Assembly MIPS e o ambiente de simulação MARS. O sistema opera como um dispositivo embarcado de segurança, implementando funcionalidades de autenticação por senha, configuração dinâmica de credenciais, menus interativos e um log de auditoria baseado em vetores circulares. A implementação prioriza a gestão manual de memória, o uso eficiente de registradores e a manipulação da pilha para preservação de contexto, demonstrando a aplicação prática dos conceitos de arquitetura de computadores em um cenário de baixo nível.

Palavras-chave: Assembly MIPS, Simulador MARS, Gestão de Memória, Autenticação, Sistemas Embarcados.

2. Introdução

O presente relatório descreve o desenvolvimento do projeto da disciplina de Arquitetura e Organização de Computadores, ministrada pela Profa. Denise Stringhini na Universidade Federal de São Paulo (UNIFESP). O objetivo central do trabalho é a implementação de um **Simulador de Dispositivo Embarcado/IoT** utilizando a linguagem de montagem (*Assembly*) na arquitetura MIPS, executado através do simulador MARS.

2.1. Contextualização e Problemática

A programação em nível de montagem impõe desafios distintos daqueles encontrados em linguagens de alto nível. Enquanto linguagens modernas abstraem o gerenciamento de memória e o fluxo de controle, o desenvolvimento em Assembly exige que o programador gerencie explicitamente o uso de registradores, a alocação de memória estática e dinâmica, e a preservação de contexto durante chamadas de subrotinas.

A **problemática** deste projeto reside na implementação de lógicas de segurança e auditoria — típicas de sistemas operacionais ou *firmwares* complexos — dentro das restrições de um ambiente de baixo nível. Operações aparentemente triviais, como a comparação de *strings* (senhas), a criação de vetores de histórico e a implementação de menus interativos, não possuem bibliotecas nativas no MIPS, exigindo a construção de algoritmos manuais byte a byte e manipulação direta de endereços de memória.

2.2. Solução Proposta

Para atender a esses requisitos, o grupo desenvolveu um **Simulador de Cofre Eletrônico**. Este sistema simula um dispositivo de controle de acesso robusto, incorporando funcionalidades como:

- **Configuração Dinâmica:** Definição e confirmação de senha mestra em tempo de execução;
- **Autenticação Segura:** Validação de credenciais via comparação de *buffers* de memória;
- **Auditoria:** Implementação de um *log* de tentativas utilizando vetores e lógica de *buffer* circular;
- **Segurança Ativa:** Sistema de bloqueio automático contra ataques de força bruta.

O projeto demonstra a aplicação prática de conceitos fundamentais da arquitetura de computadores, com ênfase no uso correto da pilha (*stack*) para preservação de registradores *\$s*, manipulação de vetores (*arrays*) e modularização de código através de chamadas de função (*jal/jr*).

3. Metodologia

O desenvolvimento do projeto foi estruturado com foco na compreensão prática da arquitetura de computadores, utilizando ferramentas de simulação para validar a lógica de baixo nível. A metodologia foi dividida entre o ambiente de desenvolvimento, a linguagem utilizada e as estratégias de implementação.

3.1. Ambiente de Desenvolvimento e Ferramentas

Para a execução e testes do código, utilizou-se o simulador **MARS (MIPS Assembler and Runtime Simulator)**, versão 4.5. O MARS foi a ferramenta escolhida por permitir:

- **Visualização em Tempo Real:** O monitoramento do estado dos 32 registradores de propósito geral e dos registradores especiais (*PC*, *HI*, *LO*) a cada ciclo de instrução.
- **Mapeamento de Memória:** A inspeção direta dos segmentos de dados (*.data*) para verificar a alocação de vetores e *buffers*, e do segmento de texto (*.text*) para analisar o código de máquina gerado.
- **Emulação de Periféricos:** O uso de *System Calls* (*syscall*) para simular a interação de entrada e saída (teclado e console) típica de um sistema embarcado.

3.2. Linguagem e Arquitetura

O projeto foi integralmente codificado em **Assembly MIPS32**, seguindo o padrão da arquitetura RISC (*Reduced Instruction Set Computer*). A escolha dessa linguagem impôs restrições técnicas que guiaram a lógica de programação:

- **Gerenciamento Manual de Recursos:** Diferente de linguagens de alto nível, não houve uso de coleta de lixo (*garbage collection*) ou abstrações de variáveis. Toda a manipulação de dados exigiu o carregamento explícito da memória para registradores (*lw*, *lb*) e o armazenamento de volta à memória (*sw*, *sb*).
- **Convenção de Chamadas:** Para garantir a modularidade, o código respeitou estritamente as convenções de registradores da arquitetura MIPS, distinguindo entre registradores temporários (*\$t0-\$t9*), que não garantem persistência, e registradores salvos (*\$s0-\$s7*), cuja preservação foi assegurada via manipulação da Pilha (*Stack*).

3.3. Estratégia de Implementação

A construção do código seguiu uma abordagem incremental e modular:

1. **Validação de Strings:** Implementação inicial da lógica byte a byte para comparação de senhas (autenticação).
2. **Controle de Fluxo:** Criação do laço principal (*Main Loop*) e do sistema de menus.
3. **Gestão de Memória:** Implementação do *buffer* circular para o histórico de auditoria e da lógica de sobrescrita de memória para a funcionalidade de troca de senha.
4. **Depuração (*Debugging*):** Testes realizados passo a passo (*step-by-step*) no MARS para identificar e corrigir erros de desvio condicional (*branches*) e estouro de pilha.

4. Descrição da Lógica e Implementação

O sistema foi estruturado como uma máquina de estados finitos, onde o fluxo de execução transita entre os modos de configuração, operação normal (menu), troca de credenciais e bloqueio de segurança. A lógica foi implementada manipulando diretamente os endereços de memória e registradores da arquitetura MIPS.

4.1. Inicialização e Configuração Segura

Diferente de sistemas com credenciais *hardcoded* (fixas no código), o simulador inicia exigindo a definição de uma senha mestra pelo usuário.

- **Lógica de Confirmação:** O sistema lê duas entradas consecutivas (*senha_secreta* e *buffer_confirmacao*). Utiliza-se uma sub-rotina de comparação byte a byte para garantir a integridade. Se houver divergência,

um laço condicional (**branch**) força o reinício do processo, impedindo que o sistema opere com uma senha desconhecida.

4.2. Comparação de Strings (Algoritmo de Autenticação)

Como a arquitetura MIPS não possui instruções nativas para comparação de *strings* completas, foi desenvolvido um algoritmo iterativo manual (**comparar_strings_auxiliar**):

1. **Ponteiros:** Carregam-se os endereços base das duas *strings* nos registradores **\$t0** e **\$t1**.
2. **Iteração:** Um contador (**\$t2**) controla um laço de 4 repetições (tamanho da senha).
3. **Verificação Byte a Byte:** A cada ciclo, utiliza-se a instrução **lb** (*Load Byte*) para extrair um caractere de cada *buffer*. Uma comparação imediata **bne** (*Branch Not Equal*) interrompe a execução ao primeiro sinal de divergência, retornando **0** (Falso).
4. **Sucesso:** Se o laço for concluído sem interrupções, a função retorna **1** (Verdadeiro).

4.3. Log de Auditoria (Vetor Circular)

Para atender ao requisito de uso de vetores, implementou-se um sistema de *log* que armazena os resultados das últimas 5 tentativas de acesso (0 para falha, 1 para sucesso).

- **Endereçamento Indexado:** O acesso ao vetor historico não é sequencial simples. O endereço de memória alvo é calculado dinamicamente pela fórmula:
$$\$Endereço = Base + (Índice \times 4) \$$$
A multiplicação por 4 é necessária, pois cada registro é uma palavra (word) de 32 bits.
- **Buffer Circular:** Para evitar o estouro de memória (*overflow*) e manter apenas os registros recentes, o índice é incrementado a cada gravação. Um comparador verifica se o índice atingiu o limite (5); se positivo, o índice é resetado para 0, sobrescrevendo a entrada mais antiga (lógica FIFO - *First In, First Out*).

4.4. Troca de Senha com Sobrescrita de Memória

A funcionalidade de alteração de senha implementa uma verificação de três etapas (Senha Atual \$to\$ Nova Senha \$to\$ Confirmação). A persistência da nova senha é realizada pela função **copiar_senha_nova**.

- Esta função utiliza a instrução **sb** (*Store Byte*) dentro de um laço para transferir, byte a byte, o conteúdo do **buffer_confirmacao** para o endereço de memória da **senha_secreta**. Isso demonstra a capacidade de manipulação destrutiva de memória para atualização de variáveis de estado.

4.5. Mecanismo de Bloqueio (Anti-Brute Force)

A segurança ativa é controlada pelo registrador `$s2`, escolhido por ser um registrador *Saved* (preservado entre chamadas).

- A cada falha de autenticação (retorno 0 da função de validação), o contador é incrementado (`addi $s2, $s2, 1`).
- Uma verificação constante compara `$s2` com o limite predefinido (5). Ao atingir o limite, o fluxo do programa é desviado incondicionalmente para o rótulo `sistema_bloqueado`, encerrando a execução (`syscall 10`) e impedindo novas tentativas.

5. Máquina de Estados

5.1. Estados do Sistema e Transições

O simulador foi projetado como uma Máquina de Estados Finitos (FSM), onde o sistema assume comportamentos distintos baseados na interação do usuário e nas variáveis de memória. Abaixo, detalhamos cada estado operacional e os eventos gatilho para suas transições.

Descrição dos Estados

- **ESTADO 01: Inicialização (Fase de Cadastro)**
 - **Descrição:** É o estado mandatório ao iniciar o sistema. O cofre não possui uma senha padrão *hardcoded*; ele exige que o usuário defina uma credencial de segurança.
 - **Comportamento:** O sistema solicita a entrada de uma nova senha e, em seguida, uma confirmação.
- **ESTADO 02: Menu Principal (Operação Normal/Idle)**
 - **Descrição:** Estado de espera onde o sistema aguarda um comando do usuário. É o ponto central de convergência do fluxo.
 - **Comportamento:** Exibe as opções disponíveis ([1] Abrir e [2] Mudar Senha) e lê um número inteiro para decidir o próximo passo.
- **ESTADO 03: Modo Abrir Cofre (Autenticação)**
 - **Descrição:** Estado transitório onde ocorre a validação de segurança para acesso ao conteúdo protegido.
 - **Comportamento:** Solicita a entrada da senha, compara com a credencial salva na memória, registra o resultado no histórico e decide se libera o acesso ou contabiliza um erro.
- **ESTADO 04: Modo Mudar Senha (Configuração)**
 - **Descrição:** Estado crítico que permite a atualização da credencial de acesso.
 - **Comportamento:** Implementa uma verificação de três etapas: validação da senha atual (para autorização), leitura da nova senha e confirmação da nova senha.
- **ESTADO 05: Sistema Bloqueado (Bloqueio de Segurança)**
 - **Descrição:** Estado terminal de segurança ativa.
 - **Comportamento:** O sistema exibe uma mensagem de violação e encerra a execução (`syscall 10`), impedindo qualquer nova interação.

Regras de Transição

As mudanças entre os estados descritos acima ocorrem mediante as seguintes condições lógicas:

1. **Validação de Cadastro (Cadastro $\rightarrow$ Menu):** Ocorre somente se o `buffer_senha` for idêntico ao `buffer_confirmacao`. Caso contrário, o sistema permanece em *loop* no estado de Cadastro.
2. **Seleção de Operação (Menu $\rightarrow$ Abrir ou Mudar):** Determinada pela entrada numérica do usuário (1 ou 2). Entradas inválidas mantêm o sistema no Menu.
3. **Erro Tolerável (Abrir/Mudar $\rightarrow$ Menu):** Se a senha digitada for incorreta, o contador de erros (`$s2`) é incrementado. Se `$s2 < 5`, o sistema retorna ao Menu para nova tentativa.
4. **Bloqueio por Força Bruta (Abrir/Mudar $\rightarrow$ Bloqueado):** Se a senha for incorreta e o contador de erros atingir o limite (`$s2 == 5`), ocorre a transição irreversível para o bloqueio.
5. **Sucesso na Operação (Abrir $\rightarrow$ Fim ou Mudar $\rightarrow$ Menu):**
 - No modo *Abrir*: A senha correta encerra o simulador com mensagem de sucesso.
 - No modo *Mudar Senha*: A troca bem-sucedida atualiza a memória e retorna ao Menu Principal com o contador de erros zerado.

5.2. Diagrama de Estados

Abaixo é apresentado o diagrama visual que ilustra o fluxo lógico do simulador de cofre eletrônico, representando os estados principais do sistema e as condições que provocam as transições entre eles (as setas), conforme detalhado nas descrições técnicas anteriores.

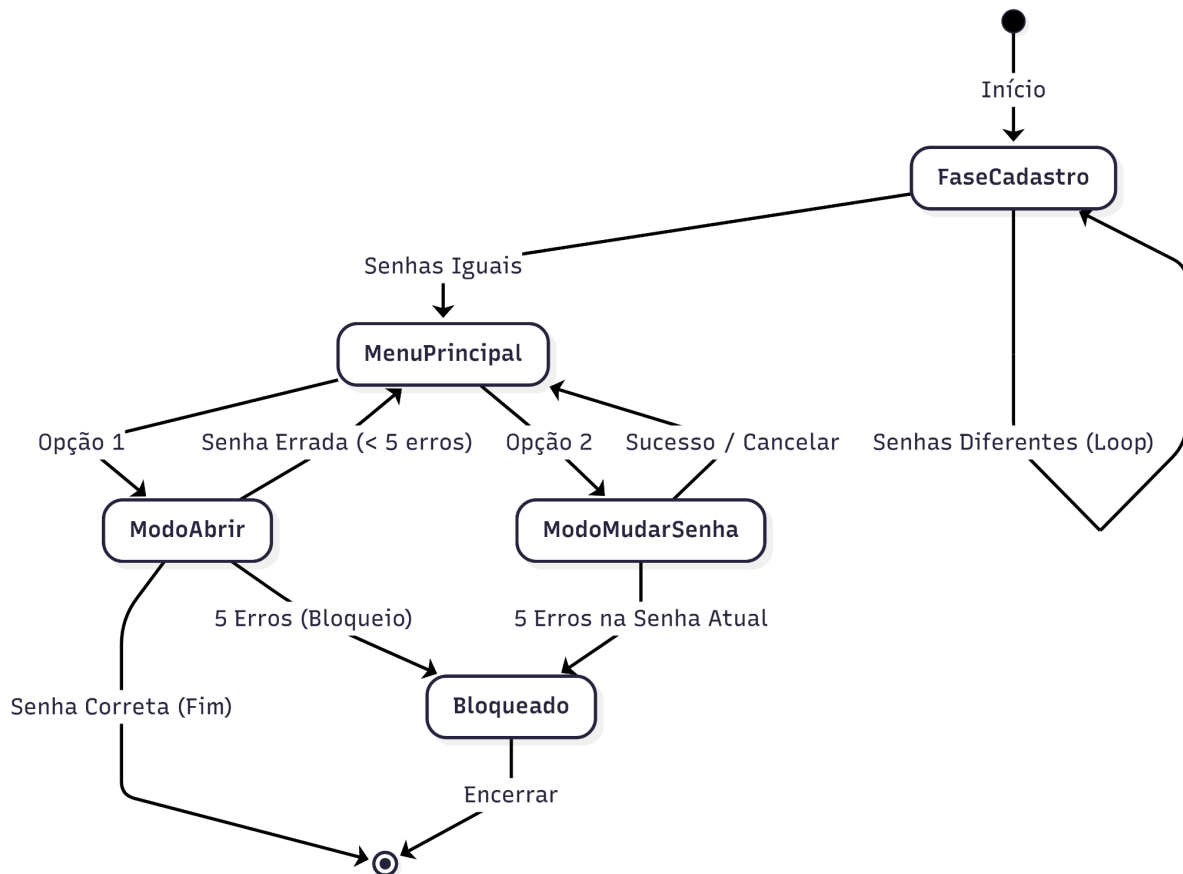


Figura 1: Diagrama de Máquina de Estados Finitos (FSM) do Simulador de Cofre, demonstrando as transições entre os modos de operação, autenticação e bloqueio de segurança.

6. Estruturas de Dados

A organização dos dados na memória do simulador foi projetada para otimizar o acesso e garantir a integridade das informações críticas (senhas) e dos registros de auditoria (histórico). Todos os dados foram alocados estaticamente no segmento `.data`.

6.1. Buffers de Caracteres (Strings)

Para a manipulação das credenciais de acesso, foram alocados três blocos contíguos de memória, tratados como vetores de bytes (*byte arrays*):

- **senha_secreta (20 bytes)**: Armazena a credencial "mestra" ativa no sistema. É a variável mais crítica do projeto, sendo escrita apenas na fase de cadastro ou na função de troca de senha.
- **buffer_entrada (20 bytes)**: Atua como uma variável volátil para capturar o *input* do usuário durante a tentativa de abertura. Seu conteúdo é sobrescrito a cada nova tentativa.
- **buffer_confirmacao (20 bytes)**: Utilizado temporariamente durante as operações de cadastro e troca de senha para garantir a verificação dupla.

A organização desses dados segue o padrão ASCII, onde cada caractere ocupa 1 byte (8 bits), permitindo a manipulação individual via instruções **lb** (*Load Byte*) e **sb** (*Store Byte*).

5.2. Vetor de Histórico (Log de Auditoria)

A principal estrutura de dados complexa do projeto é o vetor **historico**.

- **Definição:** Um vetor unidimensional de inteiros (**.word**), contendo 5 posições inicializadas com 0.
- **Representação:** Cada posição armazena um valor booleano lógico: **1** (Sucesso/Abertura) ou **0** (Falha/Erro).
- **Acesso à Memória:** Diferente dos buffers de caracteres, este vetor armazena palavras de 32 bits (4 bytes). Isso impõe que o acesso à memória respeite o alinhamento de palavras.

6.3. Convenção de Endereço Base + Deslocamento

Para acessar elementos específicos do vetor **historico**, o sistema não utiliza endereços fixos (*hardcoded*), mas sim o cálculo dinâmico de endereço efetivo. A fórmula utilizada para localizar o elemento **\$i\$** é:

$$\$ \$ \text{Endereço_Efetivo} = \text{Endereço_Base} + (\text{Índice} \times \text{Tamanho_Palavra}) \$ \$$$

No contexto do MIPS:

1. Carrega-se o **Endereço Base** (rótulo **historico**) em um registrador.
2. Carrega-se o **Índice** atual (variável **indice_hist**) em outro registrador.
3. Calcula-se o **Deslocamento** (*Offset*) multiplicando o índice por 4 (via instrução **mul** ou **sll**), visto que cada inteiro ocupa 4 bytes.
4. Soma-se o Base + Deslocamento para obter o ponteiro final para a operação **sw** (*Store Word*).

6.4. Representação de Sensores e Atuadores

Neste simulador, os dispositivos de I/O são abstraídos via chamadas de sistema (*Syscalls*):

- **Sensor de Teclado:** Representado pelas chamadas **syscall 5** (leitura de inteiros para menu) e **syscall 8** (leitura de strings para senhas).
- **Atuador de Display:** Representado pela chamada **syscall 4**, que envia sequências de caracteres ASCII para o console, simulando o display LCD do cofre.

7. Funções Principais

A arquitetura do software foi modularizada para garantir a reutilização de código e facilitar a depuração. Abaixo descrevemos as sub-rotinas essenciais que compõem o núcleo lógico do sistema.

7.1. Função: **comparar_strings_auxiliar**

Esta é a função central de autenticação, responsável por verificar a igualdade entre duas credenciais.

- **Parâmetros de Entrada:**
 - **\$t0**: Endereço de memória da primeira string (ex: **senha_secreta**).
 - **\$t1**: Endereço de memória da segunda string (ex: **buffer_entrada**).
- **Registradores Usados:**
 - **\$t2**: Contador de iterações (fixado em 4 ciclos).
 - **\$t3**: Armazena temporariamente o byte lido da primeira string.
 - **\$t4**: Armazena temporariamente o byte lido da segunda string.
- **Uso da Pilha:** Não utiliza pilha (*Leaf Function*), pois não realiza chamadas aninhadas.
- **O que Retorna:**
 - **\$v0**: Retorna **1** se as strings forem idênticas ou **0** se houver divergência.
- **Lógica Geral:**

A função executa um laço de repetição de 4 iterações. Em cada passo, carrega um byte de cada endereço de origem (**lb**), compara os valores (**bne**) e, se forem iguais, incrementa os ponteiros de endereço. Se qualquer par de bytes for diferente, a função encerra prematuramente retornando 0. Se o laço terminar, retorna 1.

7.2. Função: **registrar_historico**

Responsável pela gestão do log de auditoria circular. Esta função foi parcialmente gerada com auxílio de IA e revisada pelo grupo.

- **Parâmetros de Entrada:**
 1. **\$a0**: O estado da tentativa (0 para erro, 1 para sucesso).
- **Registradores Usados:**
 1. **\$s0**: Registrador salvo utilizado para calcular o endereço final de gravação.
 2. **\$t0, \$t1**: Manipulação do índice atual.
 3. **\$t2**: Cálculo do deslocamento (offset).
- **Uso da Pilha: Sim.**
 1. Executa **addi \$sp, \$sp, -4** e **sw \$s0, 0(\$sp)** no início para preservar o contexto do registrador **\$s0**.
 2. Executa **lw** e **addi \$sp** no final para restaurar o estado original.
- **O que Retorna:** Não possui retorno explícito (função **void**), mas altera o estado da memória RAM.

- **Lógica Geral:**

1. Lê o índice atual da memória.
2. Calcula o endereço físico ($\text{Base} + \text{Índice} * 4$).
3. Grava o valor de `$a0` na posição calculada.
4. Incrementa o índice.
5. Verifica se o índice atingiu o limite (5). Se sim, reseta para 0 (Lógica Circular FIFO).
6. Salva o novo índice atualizado na memória.

7.3. Função: `copiar_senha_nova`

Função crítica usada na funcionalidade de troca de senha para persistir a nova credencial.

- **Parâmetros de Entrada:**

- Os endereços de origem (`buffer_confirmacao`) e destino (`senha_secreta`) são carregados internamente via instrução `la`.

- **Registradores Usados:**

- `$t0`: Ponteiro de origem.
- `$t1`: Ponteiro de destino.
- `$t3`: Byte de transferência.

- **Uso da Pilha:** Não utiliza.

- **O que Retorna:** Não possui retorno, mas modifica permanentemente a variável global `senha_secreta`.

- **Lógica Geral:**

Realiza uma cópia destrutiva de memória. Um laço percorre o buffer de origem byte a byte, lendo com `lb` e escrevendo imediatamente no destino com `sb` (Store Byte), sobrescrevendo a senha antiga.

7.4. Função: `validar_senha` (Wrapper)

Atua como uma interface simplificada para a comparação.

- **Parâmetros de Entrada:** Nenhum explícito (usa as variáveis globais padrão).

- **Registradores Usados:** `$t0`, `$t1` (para configuração de argumentos).

- **Uso da Pilha:** Não.

- **O que Retorna:** `$v0` (Resultado da validação).

- **Lógica Geral:**

Prepara os argumentos carregando os endereços de `senha_secreta` e `buffer_entrada` nos registradores corretos e realiza um desvio (jump) para a lógica de `comparar_strings_auxiliar`. Funciona como uma camada de abstração para limpar o código na função `main`.

8. Trechos gerados pela IA e explicações

Para a implementação do sistema de **Log de Auditoria**, utilizamos a IA para gerar a estrutura base da função `registrar_historico`. O objetivo era criar um algoritmo eficiente para armazenar dados em um vetor circular, respeitando as convenções de chamada da arquitetura MIPS.

Abaixo, detalhamos a lógica técnica de cada bloco gerado:

Trecho Gerado (Função <code>registrar_historico</code>)	Nossa Explicação Técnica (Obrigatória)
Cálculo de Endereço Indexado (Instrução <code>mul \$t2, \$t1, 4</code>)	O que faz: Calcula o deslocamento (<i>offset</i>) exato na memória para gravar o registro atual. Nossa Análise: O MIPS endereça a memória byte a byte. Como declaramos o vetor <code>historico</code> como <code>.word</code> (inteiros de 4 bytes), não podemos somar o índice (0, 1, 2) diretamente ao endereço base. A instrução <code>mul</code> converte o índice lógico em deslocamento físico (ex: índice 2 $\times$ 4 bytes = 8 bytes de deslocamento). Somamos esse resultado ao endereço base em <code>\$s0</code> para acessar a posição correta.
Proteção de Contexto com Pilha (Instruções <code>addi \$sp, \$sp, -4</code> e <code>sw \$t0, 0(\$sp)</code>)	O que faz: Preserva o valor do registrador <code>\$s0</code> antes de utilizá-lo na função. Nossa Análise: Pela convenção do MIPS, registradores do tipo <code>\$s</code> (Saved) devem ter seus valores preservados por quem chama a sub-rotina (<i>Callee-Saved</i>). A IA gerou o código para decrementar o ponteiro da pilha (<code>\$sp</code>) em 4 bytes e salvar (<code>sw</code>) o valor antigo de <code>\$s0</code> . Ao final, o

	valor é restaurado (lw) e a pilha devolvida, garantindo que a função main não perca seus dados críticos.
Lógica de Buffer Circular (Comparação bne com limite 5)	O que faz: Implementa a rotatividade do histórico, impedindo estouro de memória. Nossa Análise: O código verifica se o índice do vetor chegou a 5 (limite máximo). Se sim, uma instrução li \$t1, 0 força o índice a voltar para o início. Isso cria um comportamento FIFO (<i>First-In, First-Out</i>), onde as novas tentativas de senha sobrescrevem automaticamente as mais antigas, mantendo o vetor sempre atualizado sem necessidade de deslocar dados manualmente.

O que tivemos que corrigir no código gerado pela IA

Embora a lógica matemática estivesse correta, realizamos os seguintes ajustes para adequação ao simulador MARS:

1. **Ajuste de Labels:** A IA sugeriu nomes genéricos para os *labels* de desvio. Renomeamos para **salva_ind** para manter o padrão de nomenclatura do restante do projeto e facilitar a leitura do fluxo.
2. **Validação de Escrita:** Adicionamos comentários e verificamos via *Data Segment* do MARS se os valores 0 e 1 estavam sendo gravados nos endereços de memória corretos (alinhados de 4 em 4 bytes), confirmando que a multiplicação do índice estava funcionando como esperado.

9. Testes e Resultados

A validação do simulador foi realizada através de uma bateria de testes manuais executados no ambiente MARS, visando cobrir o fluxo normal de operação, o tratamento de erros e os limites estruturais do sistema.

9.1. Casos de Teste Utilizados

Para garantir a robustez do código, foram definidos três cenários principais de validação:

- **Caso Típico (Fluxo Feliz):**

- *Cenário*: O usuário cadastra uma senha inicial (1234), confirma corretamente e acessa o Menu Principal. Em seguida, seleciona a opção "1 - Abrir Cofre", digita a senha correta e recebe a mensagem de "Acesso Permitido".
- *Resultado Esperado*: O sistema deve permitir o acesso e registrar o valor 1 no vetor de histórico.
- **Caso Adverso (Falha de Autenticação):**
 - *Cenário*: O usuário tenta abrir o cofre ou alterar a senha digitando uma credencial incorreta (0000).
 - *Resultado Esperado*: O sistema deve exibir a mensagem "[SENHA INCORRETA]", retornar ao menu principal (não permitir acesso) e incrementar o contador de erros interno (\$s2). O vetor de histórico deve registrar 0.
- **Caso Estrutural (Estado Limite - Bloqueio e Buffer Circular):**
 - *Cenário*: O usuário erra a senha propositalmente 5 vezes consecutivas. Além disso, verificamos se o vetor de histórico sobrescreve os dados antigos após a 6ª tentativa.
 - *Resultado Esperado*: Ao atingir o 5º erro, o sistema deve transitar imediatamente para o estado "BLOQUEADO" e encerrar a execução. A inspeção da memória deve mostrar que o índice do histórico retornou a 0 (comportamento circular).

9.2. Discussão dos Resultados

Os testes confirmaram que o simulador atende a todos os requisitos funcionais propostos. A lógica de comparação de *strings* mostrou-se precisa, diferenciando corretamente as credenciais byte a byte. O mecanismo de proteção contra força bruta (bloqueio após 5 erros) funcionou conforme esperado, protegendo o sistema contra tentativas exaustivas.

Uma limitação observada refere-se à volatilidade da memória: como o sistema roda na RAM simulada, todos os dados (senhas e logs) são perdidos ao reiniciar o simulador. Um ponto de melhoria seria a implementação de persistência de dados utilizando *Syscalls* de manipulação de arquivos (escrita em *.txt*), permitindo que o histórico e a senha mestra fossem recuperados em execuções futuras.

10. Conclusão

O desenvolvimento deste Simulador de Cofre Eletrônico proporcionou ao grupo uma compreensão aprofundada sobre o funcionamento interno dos computadores e os desafios da programação de baixo nível.

Aprendizados e Desafios O principal aprendizado foi o gerenciamento manual de recursos. Diferente de linguagens de alto nível, onde a alocação de memória e o controle de fluxo são abstraídos, no MIPS foi necessário controlar cada byte e cada desvio condicional. O maior desafio enfrentado foi a implementação da lógica de ponteiros para o *buffer* circular e a correta manipulação da pilha (*Stack*) para garantir que os registradores salvos (\$s) não fossem corrompidos durante chamadas de função aninhadas.

O Papel da IA A Inteligência Artificial Generativa atuou como uma ferramenta de apoio técnico, conforme permitido pelo edital. Ela foi utilizada para gerar esboços de algoritmos matemáticos específicos (como o cálculo de deslocamento do vetor) e para esclarecer dúvidas de sintaxe do MARS. No entanto, a integração desses trechos, a depuração lógica e a estruturação da máquina de estados foram realizadas inteiramente pelos membros do grupo, garantindo o domínio total sobre o código entregue.

Melhorias Futuras Para trabalhos futuros, o projeto poderia ser expandido com a utilização da ferramenta *Bitmap Display* e *Keyboard MMIO* do MARS para criar uma interface gráfica simulada, substituindo o console de texto por um painel visual com LEDs indicadores de estado (Verde/Vermelho), aproximando ainda mais a simulação de um dispositivo IoT real.

11. Referências

1. **PATTERSON, D. A.; HENNESSY, J. L.** *Computer Organization and Design: The Hardware/Software Interface*. 5. ed. Morgan Kaufmann, 2013. (A "Bíblia" do MIPS).
2. **MARS (MIPS Assembler and Runtime Simulator)**. Documentação oficial e lista de Syscalls. Disponível em: <http://courses.missouristate.edu/KenVollmar/MARS/>. Acesso em: dez. 2025.
3. **STRINGHINI, D.** Material de aula da disciplina de Arquitetura e Organização de Computadores. UNIFESP - ICT, 2025.
4. **MIPS Green Sheet**. MIPS Instruction Set Reference. Disponível em: https://inst.eecs.berkeley.edu/~cs61c/resources/MIPS_Green_Sheet.pdf. Acesso em: dez. 2025.